2017 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech)
Bloemfontein, South Africa, November 29 - December 1, 2017

# Sensitivity analysis of an outlier-aware k-means clustering algorithm

Peter O. Olukanmi
Institute for Intelligent Systems
Electrical & Electronics Engineering
University of Johannesburg.
peteroolukanmi@gmail.com

Bhekisipho Twala
Institute for Intelligent Systems
Electrical & Electronics Engineering
University of Johannesburg.
btwala@uj.ac.za

***Abstract*—K-means-sharp (k-means#), a recently introduced outlier-robust alternative to classical k-means, excludes points further than a threshold *T* from their cluster centroids, when computing new centroids. *T* is determined online as a constant multiple *q* of the estimated population standard deviation of the point-to-centroid distances. The approach preserves k-means' structure, and requires no preprocessing, user intervention or prior knowledge of the outlier rate. In pursuit of insights for optimizing the algorithm, this paper investigates the algorithm's sensitivity to variation in *q*. We study *q* values in the range 2-6, which corresponds to 95% - nearly 100% confidence in rejecting a point as an outlier. On 10 datasets tested, k-means# yields better within-cluster mean square error, for all values of *q*, thus establishing its effectiveness in producing good partitions. More interestingly, as *q* is varied, trade-offs are observed between clustering quality, outlier-detection accuracy, precision, recall and F1-score.**

***Keywords—k-means; clustering; outliers; outlier detection; sensitivity analysis***

## I. INTRODUCTION

The volume, velocity and variety of today's data universe, require that data analysis methods be automated and robust to real-world conditions. Machine learning algorithms often satisfy the requirement of automation and intelligence, but traditional algorithms assume outliers are absent. This assumption is often violated in real-world data. In the presence of outliers, such outlier-intolerant algorithms will still produce results, but the results will be wrong. One algorithm that suffers this drawback is the classical k-means [1][2] which is one of the most popular clustering techniques [3], also listed among the top 10 data mining algorithms [4]. Actually, sensitivity to outliers is a drawback shared by all least-squares method, of which k-means is one.

A simple but effective way to mitigate the misleading effects of outliers in k-means clustering, is to detect and exclude putative outliers when computing the feature means used to update centroids. This is the approach adopted in the recently introduced k-means-sharp (k-means#) algorithm [5], an outlier-robust alternative. K-means# preserves the classical k-means structure, modifying only the centroid update step. In each iteration, any point not within a distance $T$ from its cluster centroid is excluded from the computation of new centroids. The entire data are then reassigned to clusters by proximity to centroids, as in classical k-means. The modified algorithm minimizes mean within-cluster squared distance. Thus, classical k-means can be viewed as a special case in which no outliers are detected.

In the original implementation of k-means# [5], $T$ is computed automatically as $3\sigma$, where $\sigma$ is the estimated standard deviation of the population of point-to-centroid distances. For Gaussian distribution, which k-means assumes [3], this corresponds to 99.7% confidence in rejecting points as outliers. This specific implementation has been shown to improve k-means' performance.

In this paper, we investigate the effects of varying the constant coefficient $q$ of the population standard deviation ($q$ = 3 in the original implementation) used in computing $T$. We hypothesize that this value affects performance and determines the trade-offs in clustering accuracy, outlier-detection accuracy, precision and recall. This implies that varying $q$ is expected to reduce performance along some of these dimensions while increasing others. But how sensitive is the algorithm to such variation, and is any interesting trend observable? Specifically, what is the optimal value of $q$ when emphasis is on any one of these performance measures? To answer these questions, we study the algorithms performance on 10 datasets. The set consists of 2 popular datasets (*Iris* and *Ruspini*) and 4 outlier-contaminated versions of each. We compare average performances over 100 runs on each dataset, for $q$ = 2, 3, 4, 5 and 6. These $q$ values correspond to confidence levels between 95% and 100%. Although comparison with classical k-means is not the primary aim of this paper, k-means is included in our experiment to benchmark the clustering qualities produced by the different versions of k-means#.

The rest of this paper is organized as follows. Some background material is presented on the classical k-means algorithm, as well as k-means improvements i.e. existing related work and k-means#. Then, we describe our experiments and report results in Section III. These results are discussed in Section IV in relation to the questions posed in this Section I, before the paper is concluded in Section V.

978-1-5386-2313-8/17/$31.00 ©2017 IEEE

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.

## II. BACKROUND AND RELATED WORK

This section provides background on k-means clustering and improvements to the classical algorithm. It includes essential details on k-means# which constitutes the focus of this paper.

### A. Classical k-means and improvements

K-means partitions the data space into Voronoi cells. Its goal is to form disjoint groups of $n$ data points ($\mathbf{x}_1$, $\mathbf{x}_2$, … , $\mathbf{x}_n$) into $k \leq n$ sets $\{S_1, S_2, \dots, S_k\}$ that minimize the total within-cluster mean squared error. Formally, the problem is formulated as:

$$\underset{S}{argmin} \frac{1}{n} \sum_{i=1}^{k} \sum_{x \in S_i} \|x - \mu_i\|^2 \quad (1)$$

where $\mu_i$ is the mean of points in $S_i$.

K-means is first initialized by randomly selecting points to be used as initial set of cluster centroids, after which it repeats the following until convergence:

1. Assignment to clusters: assign each point to the same cluster as the nearest centroid to that point. K-means performs hard partitioning. Thus, $S_i \cap S_j = \emptyset \ \forall \ i, j \leq k$.
2. Centroids update: Compute new centroids as the means of each cluster.

$$m_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x_j \in S_i} x_j \quad (2)$$

Several techniques have been developed to handle outliers in k-means clustering. k-means++ [6] and Scalable k-means [7] employ seeding methods to prevent bad choices of initial centroids, such as outliers. Some approaches focus on detecting and removing outliers. Outlier detection methods include the method of [8] which computes a local outlier factor (LOF) for each point, based on the ratios of the local density of the area around the point and the local density of its neighbours. Papadimitriou et.al. propose a similar technique called local correlation integral (LOCI) [9], to address the difficulty of choosing the minimum number of points required to be supplied to the LOF-based approach. Several other techniques in the outlier detection class are based on k-nearest neighbor (kNN) graphs such as the methods of [10], [11] and [12].

A general drawback of these existing methods is reliance on user-supplied parameters or knowledge of the number of outliers. K-means# avoids these drawbacks. It does not rely on distance graphs and is easy to implement. Although it is threshold-based, the threshold is a single global value that is computed without user intervention. Outlier detection/avoidance is inherent to the algorithm, and is done simultaneously with the clustering process. A key attraction of k-means# is that it preserves k-means simplicity and efficiency by maintaining the basic structure, altering only the centroid update step. K-means# has been shown to improve k-means' clustering accuracy [5]. Our goal in this paper is to investigate its sensitivity to a crucial parameter used in the computing the global threshold.

### B. K-means-sharp (k-means#)

K-means# is based on the rationale that the misleading effects of outliers in classical k-means happens at the centroids update stage. K-means# detects outliers in each iteration and exempts them when computing new centroids. Being a Gaussian Mixture Model (GMM) based method, classical k-means assumes Gaussian data distribution [13][3]. It then follows that 99.7% inliers will have point-to-centroid distances within 3 standard deviations ($\sigma$) from the population mean. It has been shown that $\sigma = 1.4826\ MAD_e$ is an unbiased estimate of the population standard deviation [14][15], where $MAD_e$ = median $|e_i - \text{median}(e_i)|$, and $e_i$ is the distance of point $i$ to its centroid. Thus, the point-to-centroid distance threshold $T$ which distinguishes inliers from outliers is given by:

$$T = 3\sigma = 3(1.4826\ MAD_e)$$

$$T = 4.4478 MAD_e \quad (3)$$

Given a suitable value of $T$ computed according to equation (4), in place of the standard update formula in (3), centroids are updated as follows:

$$m_i^{(t+1)} = \frac{1}{|S_i^{(t)}| - |S_{io}^{(t)}|} \sum_{x_j \in S_i} x_j : x_j = \mathbf{0}\ if\ \|x - \mu_i\| > T \quad (4)$$

where $S_{io}^{(t)} \in S_i^{(t)}$ is the set of points $x_j$ in $S_i^{(t)}$ that have point-to-centroid distance $\|x - \mu_i\| \geq T$.

Equations (2) and (4) are similar. Both compute a new centroid as the mean of points in the same cluster. However, the difference is that in (4), each point that is not within distance $T$ from its cluster centroid, is excluded from the computation. Due to exclusion of outliers, the cost function becomes:

$$\boldsymbol{C} = \sum_i \rho\,(\boldsymbol{e_i^2}) \quad (6)$$

$$\rho(\boldsymbol{e_i^2}) = \begin{cases} \|x - \mu_i\|^2 & \|x - \mu_i\| < \boldsymbol{T} \\ 0 & \|x - \mu_i\| \geq \boldsymbol{T} \end{cases} \quad (7)$$

The modified k-means (k-means#) is outlined as follows [5]:

1. Select initial centroids as in k-means.
2. Cluster points around the current centroids, as in k-means, and compute cost $C$ using equation (6).
3. Compute $T$ using equation (4)
4. Update centroids using equation (5)
5. Stop, if maximum iterations is reached. Otherwise, go to step 2.
6. Return as final solution, the partition with minimum $C$.

## III. EXPERIMENTAL RESULTS

We study the algorithms performance on 10 datasets. The set consists of 2 popular datasets (*Iris* and *Ruspini*) and 4 outlier-contaminated versions of each. We compare average performances over 100 runs on each dataset, for $q$ = 2, 3, 4, 5 and 6. These $q$ values correspond to confidence levels between 95% and 100%. Clustering errors (mean within-cluster squared distance) are recorded in Table 1 while outlier detection accuracy, precision, recall and F1 scores are recorded in Tables 2 – 5.


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I. WITHIN-CLUSTER MEAN SQUARED ERROR (ALL ALGORITHMS ARE INITIALIZED IDENTICALLY)

| Dataset (with outlier rate in %) | k-means | k-means# (q = 2) | k-means# (q = 3) | k-means# (q = 4) | k-means# (q = 5) | k-means# (q = 6) |
|---|---|---|---|---|---|---|
| Iris (0%) | 0.78 | 0.48 | 0.51 | 0.56 | 0.61 | 0.62 |
| Iris (5%) | 2.73 | 1.31 | 1.46 | 1.35 | 1.07 | 0.91 |
| Iris (10%) | 3.25 | 1.13 | 1.68 | 2.21 | 1.93 | 1.47 |
| Iris (15%) | 3.54 | 1.63 | 1.71 | 1.53 | 1.41 | 1.61 |
| Iris (20%) | 3.16 | 2.20 | 2.44 | 1.93 | 2.12 | 2.28 |
| Ruspini (0%) | 689.82 | 384.86 | 443.51 | 395.79 | 409.27 | 417.47 |
| Ruspini (5%) | 823.58 | 470.41 | 566.61 | 573.85 | 565.06 | 557.51 |
| Ruspini (10%) | 1058.14 | 555.41 | 747.17 | 761.89 | 698.46 | 695.08 |
| Ruspini (15%) | 1104.37 | 710.59 | 879.98 | 799.46 | 757.08 | 757.06 |
| Ruspini (20%) | 1542.22 | 906.67 | 1194.91 | 1102.66 | 1102.23 | 1117.23 |

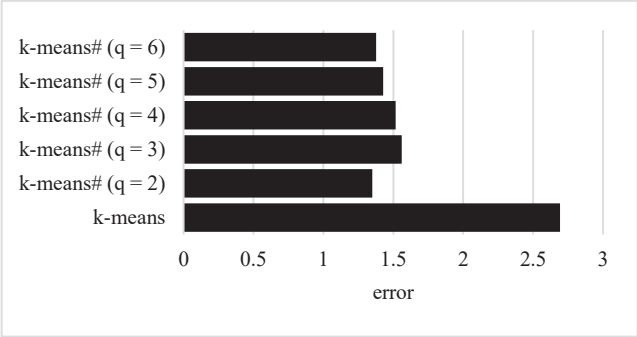



Fig. 1. Overall average of mean squared error on all Iris sets (k-means and k-means#)

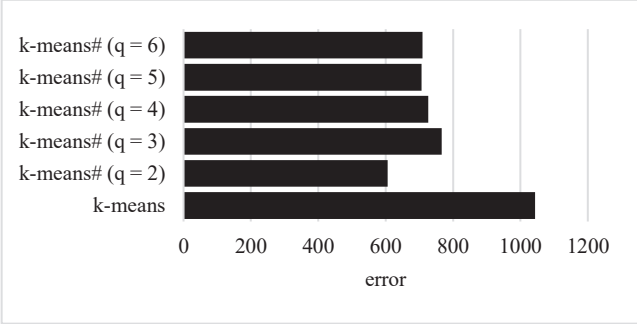



Fig. 2. Overall average of mean squared error on all Ruspini sets (k-means and k-means#)

TABLE II. OUTLIER DETECTION ACCURACY

| Dataset (with outlier rate in %) | k-means# (q = 2) | k-means# (q = 3) | k-means# (q = 4) | k-means# (q = 5) | k-means# (q = 6) |
|---|---|---|---|---|---|
| Iris (5%) | 97.1 | 97.6 | 99.5 | 96.6 | 95.6 |
| Iris (10%) | 93.8 | 95.2 | 93.4 | 92.3 | 90.7 |
| Iris (15%) | 96.9 | 91.2 | 86.7 | 88.3 | 85.7 |
| Iris (20%) | 90.9 | 85.9 | 81.8 | 80.5 | 80.0 |
| Ruspini (5%) | 42.5 | 88.4 | 95.4 | 95.4 | 95.3 |
| Ruspini (10%) | 55.2 | 88.4 | 92.1 | 90.9 | 89.7 |
| Ruspini (15%) | 75.5 | 89.1 | 86.2 | 85.4 | 85.4 |
| Ruspini (20%) | 79.9 | 83.9 | 81.0 | 80.3 | 80.2 |
| **Overall mean** | **79.0** | **90.0** | **89.5** | **88.7** | **87.8** |


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.

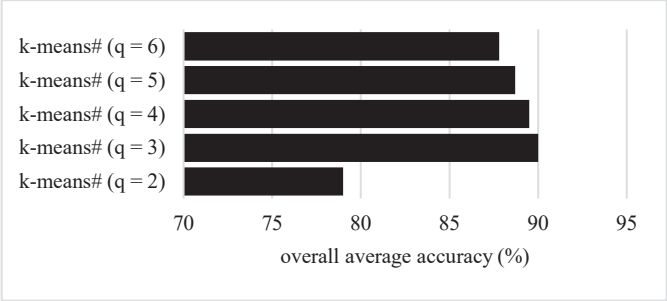



Fig. 3. Overall average outlier detection accuracy for each version of k-means#

TABLE III. OUTLIER DETECTION PRECISION

| Dataset (with outlier rate in %) | k-means# (q = 2) | k-means# (q = 3) | k-means# (q = 4) | k-means# (q = 5) | k-means# (q = 6) |
|---|---|---|---|---|---|
| Iris (5%) | 87.2 | 80.9 | 100 | 100 | 100 |
| Iris (10%) | 86.1 | 99.6 | 100 | 100 | 100 |
| Iris (15%) | 99.1 | 100 | 100 | 100 | 100 |
| Iris (20%) | 95.2 | 100 | 100 | 100 | 100 |
| Ruspini (5%) | 11.5 | 53.9 | 87.2 | 100 | 100 |
| Ruspini (10%) | 23.3 | 58.0 | 95.5 | 100 | 100 |
| Ruspini (15%) | 41.6 | 95.4 | 98.4 | 100 | 100 |
| Ruspini (20%) | 55.8 | 99.0 | 100 | 100 | 100 |
| **Overall mean** | **62.5** | **85.9** | **97.6** | **100** | **100** |

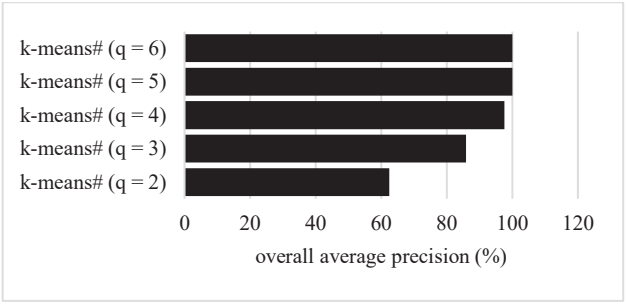



Fig. 4. Overall average outlier detection precision for each version of k-means#

TABLE IV. OUTLIER DETECTION RECALL

| Dataset (with outlier rate in %) | k-means# (q = 2) | k-means# (q = 3) | k-means# (q = 4) | k-means# (q = 5) | k-means# (q = 6) |
|---|---|---|---|---|---|
| Iris (5%) | 100 | 99.3 | 91.4 | 35.4 | 17.6 |
| Iris (10%) | 86.6 | 52.8 | 33.6 | 23.0 | 7.5 |
| Iris (15%) | 83.6 | 39.8 | 9.0 | 20.5 | 2.7 |
| Iris (20%) | 63.5 | 29.3 | 9.2 | 2.7 | 1.0 |
| Ruspini (5%) | 96.3 | 46.5 | 25.3 | 14.0 | 11.3 |
| Ruspini (10%) | 83.3 | 62.4 | 28.8 | 14.9 | 3.6 |
| Ruspini (15%) | 75.9 | 28.6 | 6.2 | 0.3 | 0.4 |
| Ruspini (20%) | 58.6 | 19.8 | 5.1 | 1.3 | 1.1 |
| **Overall mean** | **81.0** | **47.3** | **26.1** | **14.0** | **5.7** |


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.

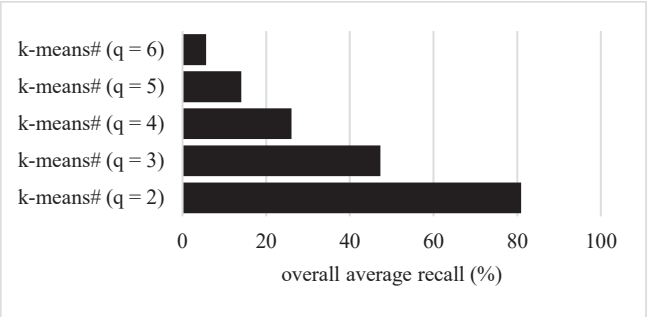



Fig. 5. Overall average outlier detection recall for each version of k-means#

TABLE V. OUTLIER DETECTION F1-SCORE

| Dataset (with outlier rate in %) | k-means# (q = 2) | k-means# (q = 3) | k-means# (q = 4) | k-means# (q = 5) | k-means# (q = 6) |
|---|---|---|---|---|---|
| Iris (5%) | 89.8 | 86.3 | 94.5 | 50.1 | 54.6 |
| Iris (10%) | 82.3 | 68.1 | 50.2 | 37.2 | 25.3 |
| Iris (15%) | 89.8 | 55.6 | 17.3 | 58.1 | 28.4 |
| Iris (20%) | 74.1 | 45.6 | 18.0 | 16.9 | 6.4 |
| Ruspini (5%) | 18.1 | 33.6 | 44.6 | 40.0 | 40.0 |
| Ruspini (10%) | 33.4 | 57.4 | 67.6 | 56.9 | 40.9 |
| Ruspini (15%) | 50.1 | 54.1 | 38.5 | 42.9 | 16.7 |
| Ruspini (20%) | 54.1 | 32.9 | 21.7 | 22.4 | 15.9 |
| **Overall mean** | **61.5** | **54.2** | **44.1** | **40.6** | **28.5** |

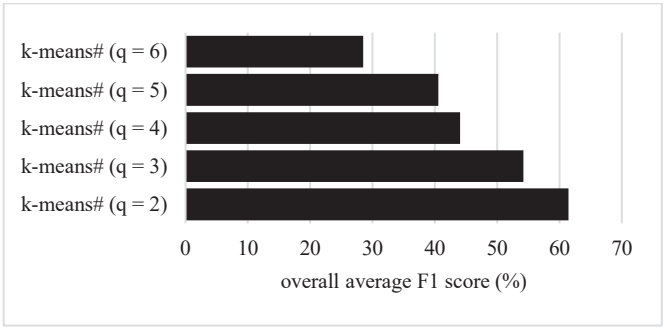



Fig. 6. Overall average outlier detection F1 score for each version of k-means#

## IV. DISCUSSION

The results presented in Section III show k-means# exhibits generally high accuracy and precision (Tables 2 and 3), with moderate recall (Table 4). High accuracy implies the algorithm performs well in classifying points as inliers or outliers, with equal emphasis on both classes. Precision measures the percentage of retrieved points that are relevant i.e. the probability that a point identified as an outlier is indeed an outlier. Recall on the other hand measures the percentage of true outliers than the algorithm identifies as outliers. F1 score combines precision and recall.

We observe some trends in the relationship between these performance metrics and $q$ (Figures 3-6). Precision increases as $q$ increases. Precision is quite high (average above 85%) for $q$ = 3; from $q$ = 5 upwards, it is 100% for every dataset tested. Recall and F1 score decrease as $q$ increases. They peak (81.0% and 61.5% respectively) at $q$ = 2. At $q$ = 6, both metrics become very low (5.7% and 28.5% respectively). Unlike the other metrics, accuracy does not trend in a single direction. It is lowest (79.0%) at $q$ = 2 and highest (90.0%) at $q$ = 3. From $q$ = 4, it decreases slowly as $q$ is increased, although staying close to 90%.

We plot the mean squared errors for the *Iris* datasets separate from the *Ruspini* sets (Figures 1 and 2), due to the difference in the range of values contained in both sets. However, both sets show similar trends. Irrespective of the values of $q$, k-means# yielded lower mean within-cluster squared distance (simply called error) than k-means. Like the other metrics, the error shows interesting trend in its relationship to $q$. Setting $q$ = 2 gives the best (lowest) error and $q$ = 3 gives the worst; as $q$ is increased further, the error decreases slowly.

We perform statistical tests to determine the significance of the difference in clustering quality (based on within-cluster errors). Analysis of variance (ANOVA) is used to test for difference among all k-means# versions while a paired 2-sample t-test is chosen as the appropriate test for 2 groups (k-means vs each k-means# version) because the datasets and initializations are identical. The outcomes are summarized as follows:


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.

*ANOVA*: Any significant difference detected among the whole group of k-means# implementations?

- All k-means# versions: $p = 0.99$ (not significant).

*Paired t-tests*: Any significant difference between classical k-means and each version of k-means#?

- k-means vs k-means# ($q = 2$): $p = 0.02$ (significant).
- k-means vs k-means# ($q = 3$): $p = 0.02$ (significant).
- k-means vs k-means# ($q = 4$): $p = 0.02$ (significant).
- k-means vs k-means# ($q = 5$): $p = 0.02$ (significant).
- k-means vs k-means# ($q = 6$): $p = 0.02$ (significant).

*Paired t-test*: Any significant difference between best k-means# and worst k-means#?

- Best k-means# ($q = 2$) vs Worst ($q = 3$): $p = 0.04$ (slightly significant)

The above tests indicate that at significance level of 0.05 (95% statistical confidence), each version of k-means# yields significantly better clusters than classical k-means. The difference is not that easy to tell when the individual versions of k-means# are compared, although the difference between the best and worst is significant.

## V. Summary and Conclusison

We performed sensitivity analysis on a simple outlier detector for k-means clustering. Introducing this detector into the classical k-means yields a robust algorithm dubbed k-means-sharp (k-means#). K-means# modifies only the centroid update step. Points not within a distance $T$ from their cluster centroids are excluded when computing new centroids. $T$ is automatically computed as a constant multiple $q$ of the estimated population standard deviation of the point-to-centroid distances. While k-means#'s effectiveness and improvement over k-means has been validated in another recent paper, this paper focuses on investigating the algorithm's sensitivity to variation in this crucial parameter $q$ to glean insight into performance trade-offs due to changes in $q$. Through the insights gleaned, the algorithm can be optimized as desired. In our experiments, we let $q$ assume the values 2, 3, 4, 5 and 6, corresponding to confidence level range 95% to near 100%.

For all values of $q$ studied, k-means# yielded lower mean within-cluster squared error than classical k-means, and the difference from k-means is significant in every case. The error shows interesting trend in its relationship to $q$. The value $q = 2$ gives the best (lowest) error while $q = 3$ gives the worst. As $q$ is increased further, the error decreases slowly.

In terms of outlier detection, precision and accuracy are generally high across the values of $q$ studied (with peaks of 100% and 90% respectively). Precision increases as $q$ increases. The algorithms is practically 100% precise at $q > 4$. Recall and F1 score are generally moderate and they decrease as $q$ increases, with peaks of 81.0% and 61.5% respectively, and minima of 5.7% and 28.5% respectively. Unlike the other outlier detection metrics, accuracy does not trend in a single direction. It is lowest (79.0%) at $q = 2$ and highest (90.0%) at $q = 3$, after which it decreases slowly as $q$ is increased, although staying close to 90%.

In summary, although the original implementation of k-means# uses $q = 3$ (99.7% confidence), this study shows that accuracy is the metric that is maximized with such a choice. However, $q = 2$ (95.4% confidence) yields best recall, F1 score and within-cluster mean squared error. Precision improves as $q$ is increased. About 98% precision is reached at $q = 4$, and higher values make the algorithm practically 100% precise. Setting $q = 2$ yields best within-cluster mean square error.

## References


[1] G. Gan, M. Kwok, and -Po Ng, "k -means clustering with outlier removal," *Pattern Recognition Letters*, vol. 90, pp. 8–14, 2017.

[2] S. Chawla and A. Gionis, "k-means–: A unified approach to clustering and outlier detection," *In Proceedings of the 2013 SIAM International Conference on Data Mining, pp. 189-197. Society for Industrial and Applied Mathematics*, 2013.

[3] G. Hamerly and C. Elkan, "Learning the k in k-means," *Advances in neural information processing*, 2004.

[4] X. Wu, V. Kumar, J. Quinlan, J. Ghosh, and Q. Yang, "Top 10 algorithms in data mining," *Knowledge and information systems*, 2008.

[5] P. O. Olukanmi and B. Twala, "K-means-sharp: modified centroid update for outlier-robust k-means clustering," in *Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech), IEEE*, 2017.

[6] Arthur, David, and Sergei Vassilvitskii. "k-means++: The advantages of careful seeding." In *Proceedings of the eighteenth annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1027-1035. Society for Industrial and Applied Mathematics, 2007.

[7] B. Bahmani, B. Moseley, A. Vattani, and R. Kumar, "Scalable k-means++," *Proceedings of the VLDB Endowment* 5, no. 7 (2012): 622-633., 2012.

[8] M. Breunig, H. Kriegel, R. Ng, and J. Sander, "LOF: identifying density-based local outliers," In *ACM sigmod record*, vol. 29, no. 2, pp. 93-104. ACM, 2000.

[9] S. Papadimitriou and H. Kitagawa, "Loci: Fast outlier detection using the local correlation integral," In P*roceedings of 19th International Conference on Data Engineering*, pp. 315-326. IEEE, 2003.

[10] M. Brito, E. Chavez, A. Quiroz, and J. Yukich, "Connectivity of the mutual k-nearest-neighbor graph in clustering and outlier detection," *Statistics & Probability Letters*, 1997.

[11] V. Hautamaki and I. Karkkainen, "Outlier detection using k-nearest neighbour graph," *Pattern Recognition*, 2004.

[12] Jin, Wen, Anthony KH Tung, Jiawei Han, and Wei Wang. "Ranking Outliers Using Symmetric Neighborhood Relationship." In *PAKDD*, vol. 6, pp. 577-593. 2006.

[13] P. Murugavel and M. Punithavalli, "Performance Evaluation of Density-Based Outlier Detection on High Dimensional Data." *International Journal on Computer Science and Engineering* 5, no. 2 (2013): 62, 2013.

[14] P. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation." *Journal of the American Statistical association* 88, no. 424 (1993): 1273-1283. 1993.

[15] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology* 49, no. 4 (2013): 764-766, 2013.




Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 15:05:37 UTC from IEEE Xplore. Restrictions apply.